

UNIT II

Problem Solving using Search & Knowledge Representation

Represent the problem → Search for a solution → Represent knowledge → Reason with knowledge

Uninformed Search: BFS, DFS | Informed Search: Best First, Hill Climbing, A* |
Heuristic functions | Semantic Networks & Frames | Production & Expert Systems |
Propositional & Predicate Logic | Bayes' Theorem

“This unit develops two major capabilities of Artificial Intelligence: solving problems through search, and representing knowledge so a system can reason with it.”

What Did We Learn in Unit I?

AI Foundations

- Introduction, history and evolution of AI
- Characteristics and types of AI
- AI, ML, DL and Data Science
- Intelligent systems and the AI lifecycle
- Real-world AI applications

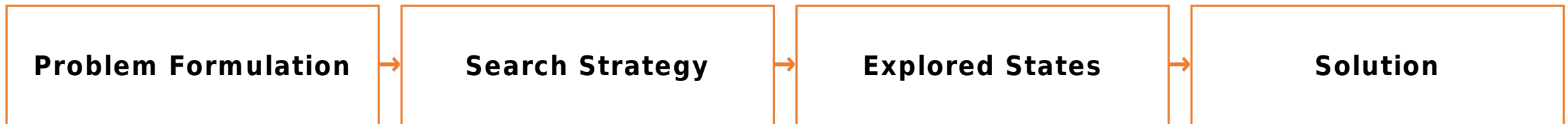
AI Problem-Solving Foundations

- Problem formulation
- Initial state and goal state
- Actions and state transitions
- State-space representation
- Search space and solution space
- Performance measures

We already know how an AI problem is represented. Now we study how the system searches for a solution.

From Problem Formulation to Search

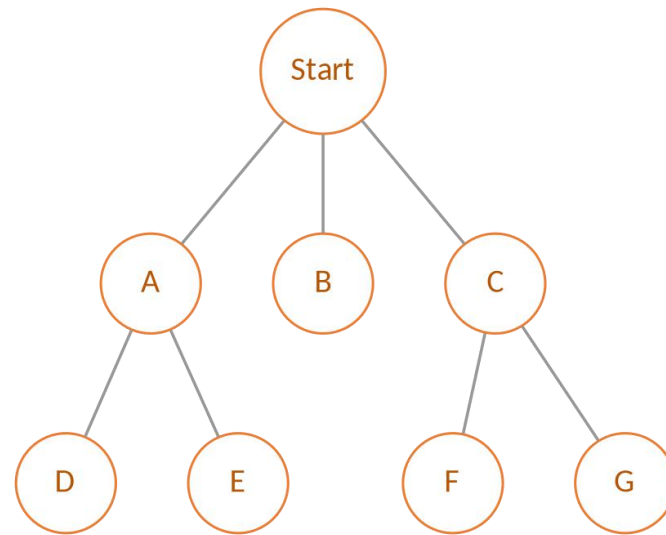
Element	Meaning
Initial State	Where the problem begins
Goal State	Desired result
Actions	Available operations
Transitions	Changes produced by actions



This representation alone does not tell the system which path to follow. The search strategy determines the order in which possible routes or states are explored.

Why Does AI Need Search?

Many AI problems do not have a single obvious path to the goal. A state may lead to several possible next states, and each of those states may generate more possibilities.



Search provides a systematic method to

- Explore possible states
- Avoid unnecessary exploration where possible
- Select the next state to examine
- Detect the goal and build a solution path

Search in Real AI Problems

Maze Solving

- 1 Explore possible movements until a valid route to the exit is found

Route Planning

- 2 Find a path between locations while considering the available routes

Robot Path Planning

- 3 Determine how a robot can move through an environment toward a destination

Puzzle Solving

- 4 Explore possible configurations until the required configuration is reached

Game Playing

- 5 Explore possible moves and resulting states to select an appropriate action

Initial State



Possible Actions



New States



Goal

A search problem works with the problem formulation already established in Unit I

CSE276

Search Problem

1 Initial State
The starting situation of the problem

2 Goal State
The condition that represents successful completion

3 Actions
Operations available to the system

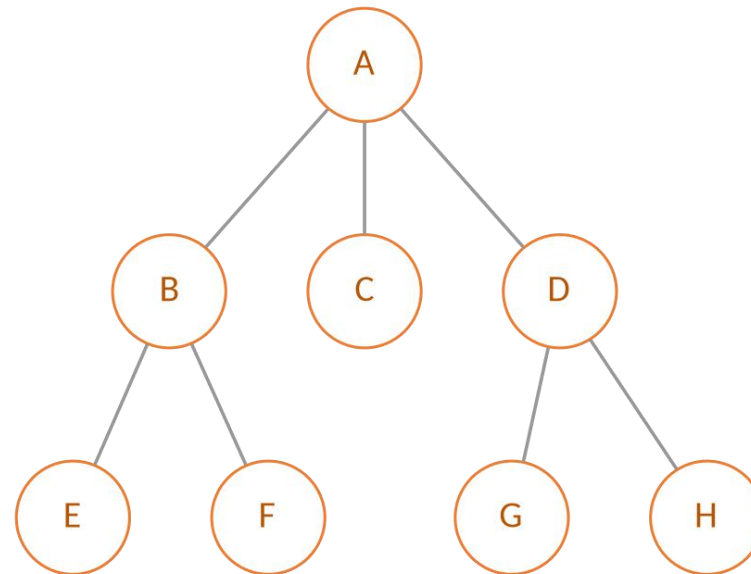
4 State Transitions
Changes produced when actions are applied

5 Goal Test
A condition used to determine whether the current state satisfies the goal

6 Solution
A valid sequence of actions from the initial state to a goal state

Search Tree

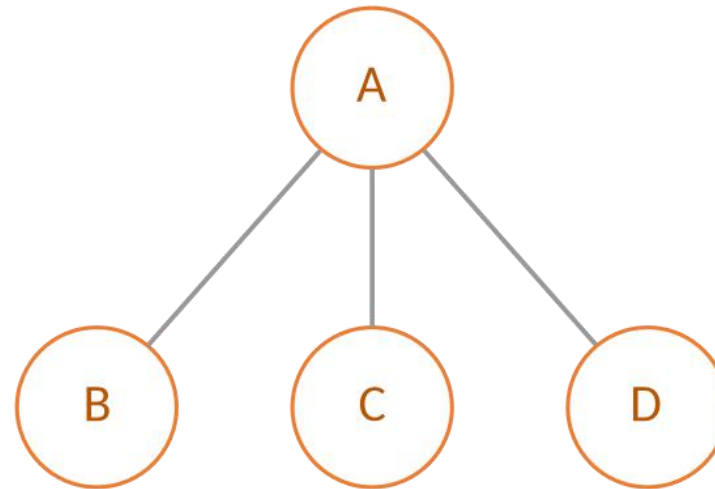
Definition: A search tree represents the states generated during problem solving.



- Root node: initial state
- Child nodes: states generated from a parent
- Branches: possible transitions
- Leaf nodes: states with no generated successors
- Goal node: state satisfying the goal condition

Node Expansion

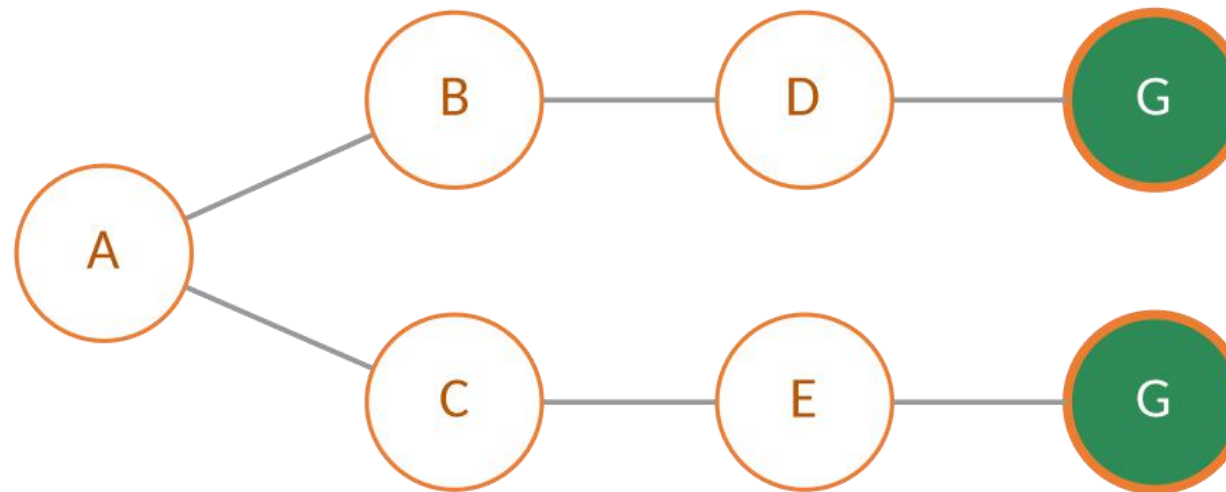
When a search algorithm examines a node and generates its successor states, the node is expanded.



Parent	Current expanded node
Child	Newly generated successor
Successor	State reached through an available action
Expansion	Generating successors from a selected node

Search Path

Definition: A path is a sequence of connected states produced by valid transitions.



Path 1

$A \rightarrow B \rightarrow D \rightarrow G$

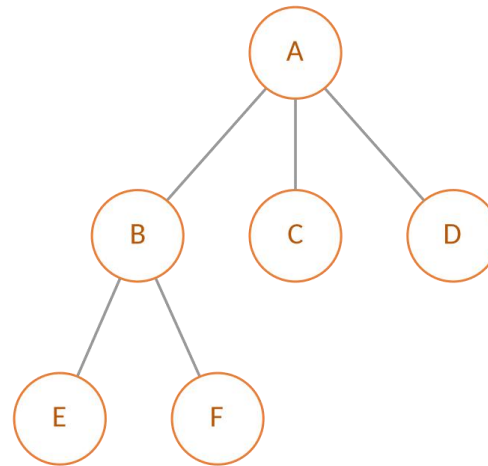
Path 2

$A \rightarrow C \rightarrow E \rightarrow G$

Both reach the same goal, but they may differ in number of steps, cost, resources required, and search effort.

Search Strategy

Definition: A search strategy determines the order in which generated states are selected for exploration.



Breadth-oriented

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Depth-oriented

$A \rightarrow B \rightarrow E \rightarrow F \rightarrow C \rightarrow D$

“A search algorithm is distinguished largely by how it chooses the next state to explore.”

Uninformed Search

Definition: Uninformed search uses only the information provided by the problem itself. It does not use an additional estimate of which state is closer to the goal.

Available information

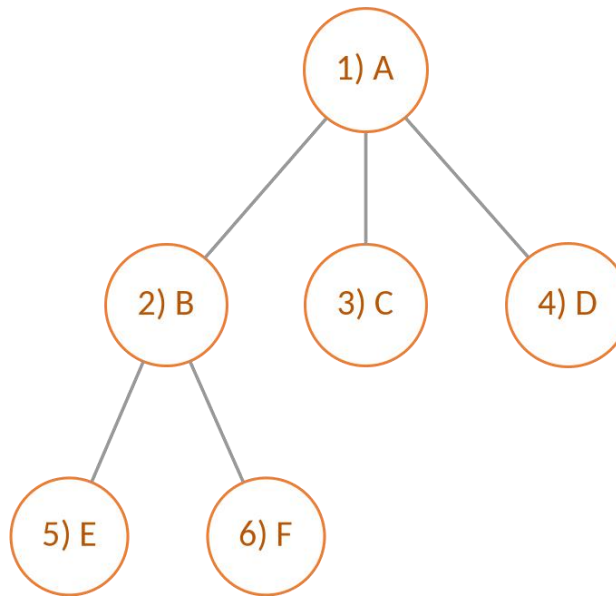
Initial state, Actions, State transitions, Goal condition

1 Breadth First Search (BFS)
Explores states level by level

2 Depth First Search (DFS)
Explores one branch deeply before moving to another

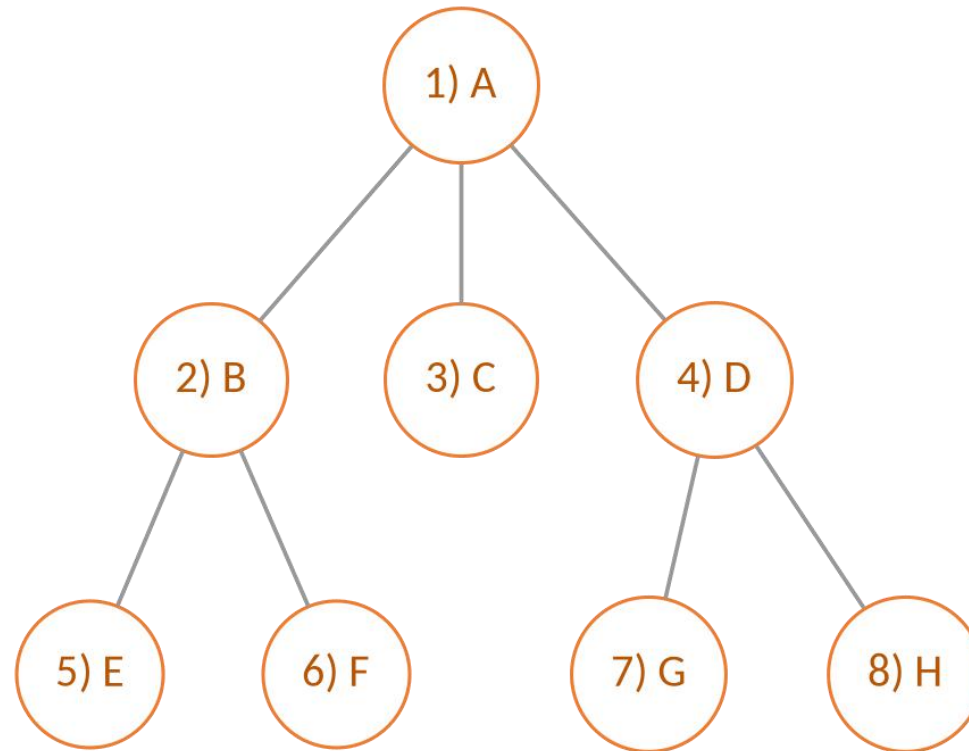
Breadth First Search

Definition: Breadth First Search is an uninformed search algorithm that explores all nodes at one depth before moving to the next depth.



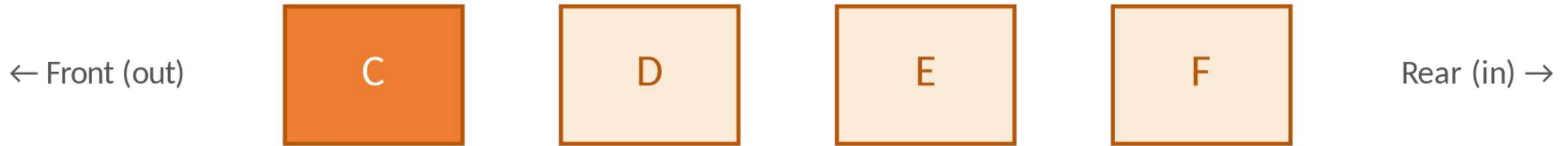
- Level-by-level exploration
- Uses a queue
- Does not use heuristic information
- Useful when shallow solutions are important

BFS Level Order



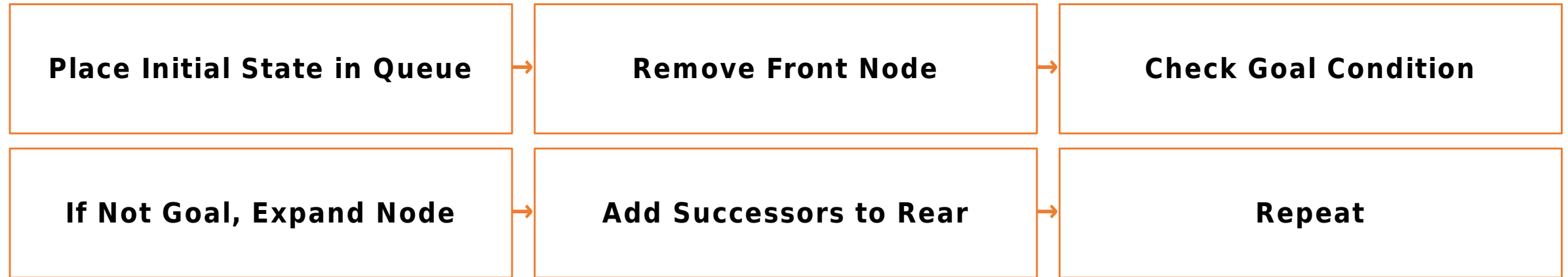
Key property: BFS can be visualized through depth levels: complete the current level before exploring the next. A node at a deeper level is not explored until the nodes at the previous level have been processed — the central behavior that distinguishes BFS from DFS.

BFS Queue



Queue principle: BFS uses a Queue to maintain the order of nodes waiting for exploration: FIFO — First In, First Out. B was inserted first, so B is removed first. After expanding B, C remains ahead of the newly added E and F, so BFS naturally explores by increasing depth.

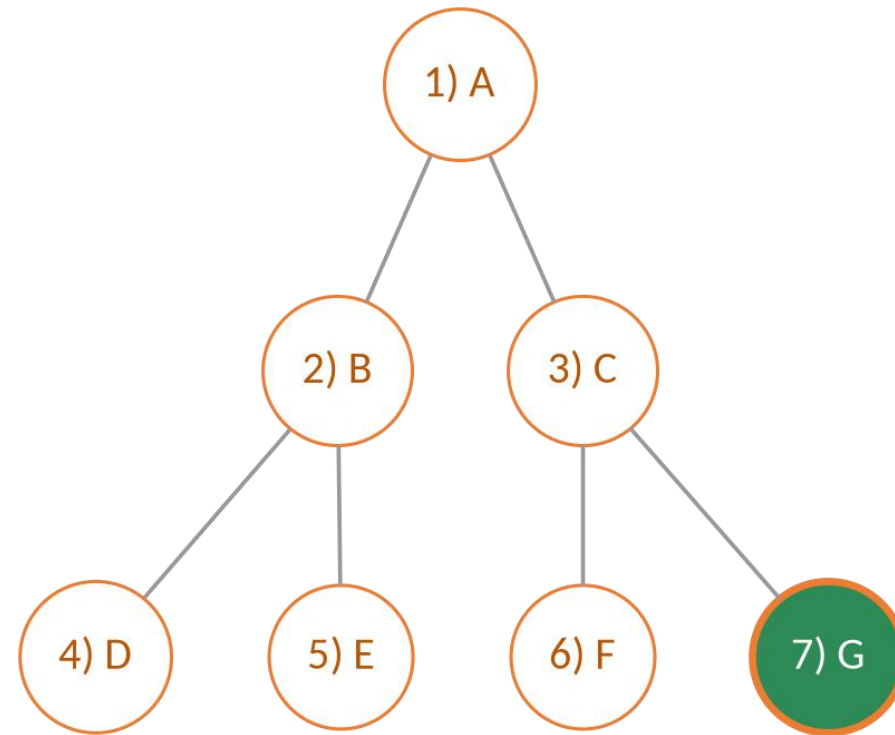
BFS Working



BFS continues until the goal is found or there are no more states to explore.

Assume G is the goal

BFS Search Tree



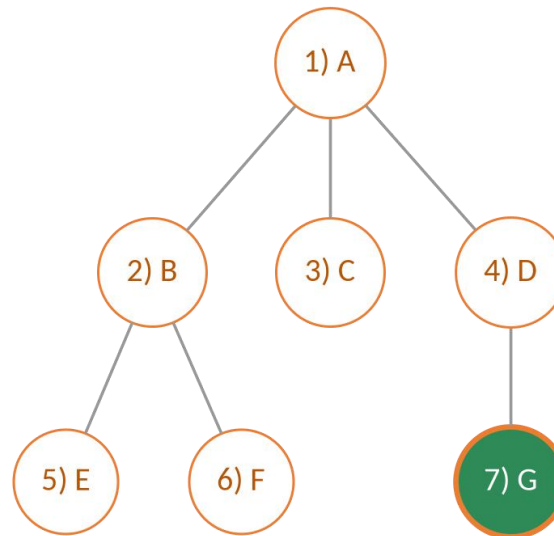
BFS does not know where the goal is. It systematically explores the search tree according to depth, reaching G only after processing the nodes that appear before it in breadth-first order.

BFS Example — Step-by-Step Traversal

Step	Queue
Start	[A]
After A	[B, C]
After B	[C, D, E]
After C	[D, E, F, G]
After D	[E, F, G]
After E	[F, G]
After F	[G]
After G	Goal Found

Traversal: A → B → C → D → E → F → G. The queue shows exactly why BFS maintains level order.

BFS Traversal Calculation



Level 0	A
Level 1	B, C, D
Level 2	E, F, G — goal G is at depth 2
Nodes selected up to the goal	7
Nodes before the goal	6

BFS Properties

1 **Completeness**

Can find a solution when one exists, under standard finite-search conditions

2 **Minimum-Depth Solution**

When every action has the same cost, BFS finds the solution with the fewest steps

3 **Exploration Order**

Nodes are explored according to increasing depth

4 **Memory Requirement**

BFS may store many frontier nodes at the same time

5 **Search Behavior**

Suitable when the expected solution is shallow and memory is sufficient

BFS Strengths and Limitations

Strengths

- Complete under standard conditions
- Finds minimum-depth solutions when step costs are equal
- Simple and systematic
- Easy to visualize and implement

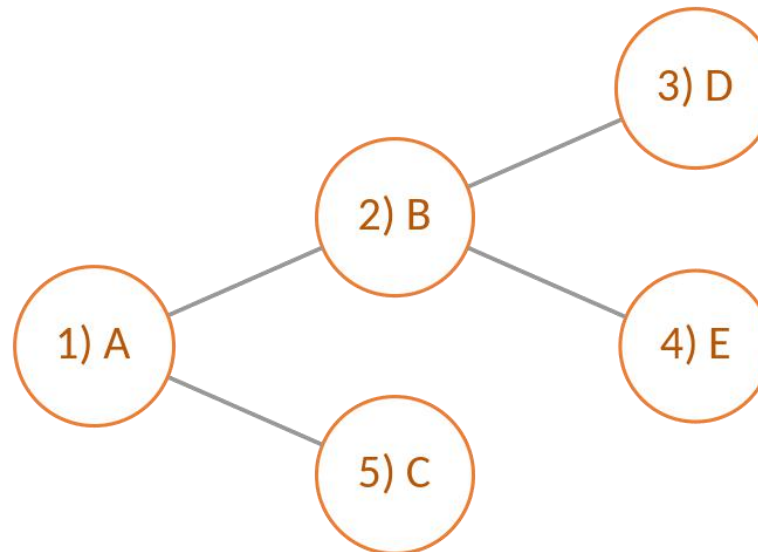
Limitations

- High memory requirement
- May expand many unnecessary nodes
- Becomes expensive as the tree grows
- Does not use heuristic information

Typical applications: Unweighted shortest-path problems, maze solving, network exploration, route exploration with equal step costs.

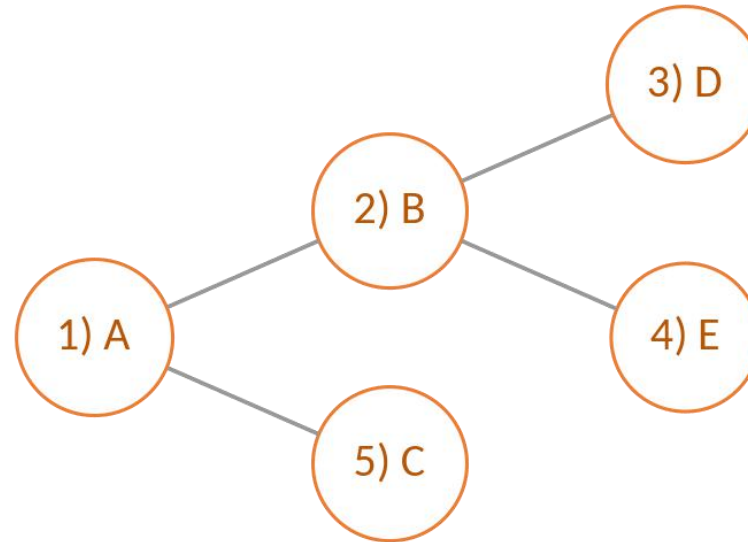
Depth First Search

Definition: Depth First Search (DFS) is an uninformed search algorithm that explores one branch as deeply as possible before backtracking and exploring another branch.



- Depth-oriented exploration
- Uses a stack or equivalent recursive mechanism
- Can reach deep solutions without storing an entire frontier level
- May explore an unsuitable branch before a better one

DFS Depth Order



BFS

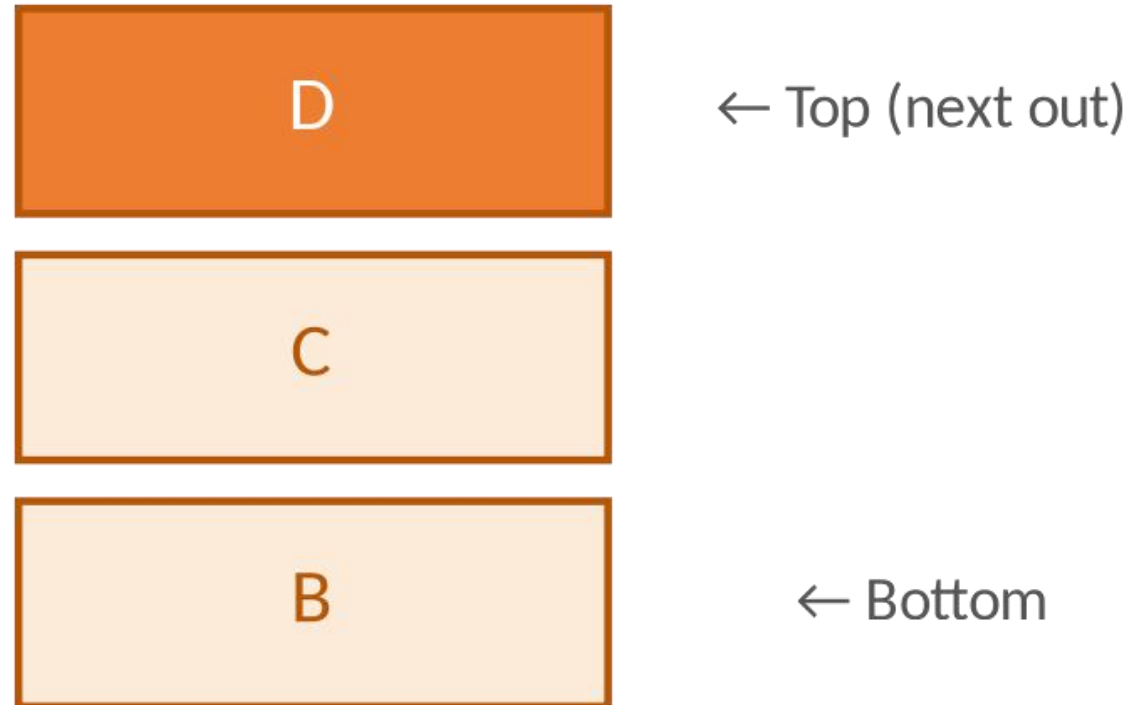
Explore nearby states first: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

DFS

Explore one path deeply first: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

This difference in exploration order is what gives BFS and DFS their different performance and solution behaviors.

DFS Stack



Stack principle: DFS uses a Stack, which follows LIFO — Last In, First Out. The most recently added node is explored first. If D is pushed after B and C, D becomes the next node removed — continue deeper before returning to other pending branches.

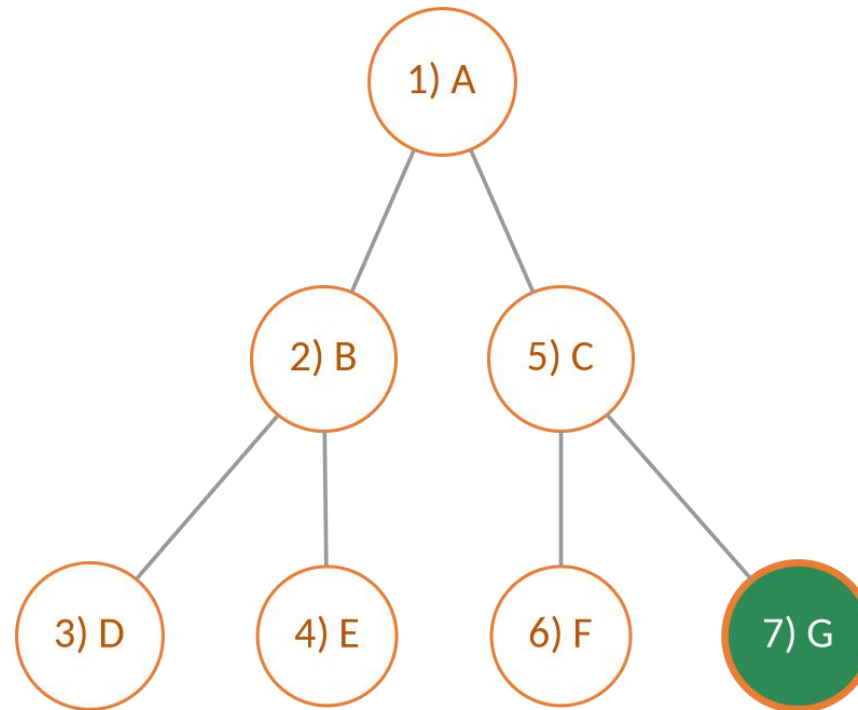
DFS Working



If a branch cannot lead to a useful continuation, DFS can backtrack and continue with another branch.

Assume G is the goal

DFS Search Tree

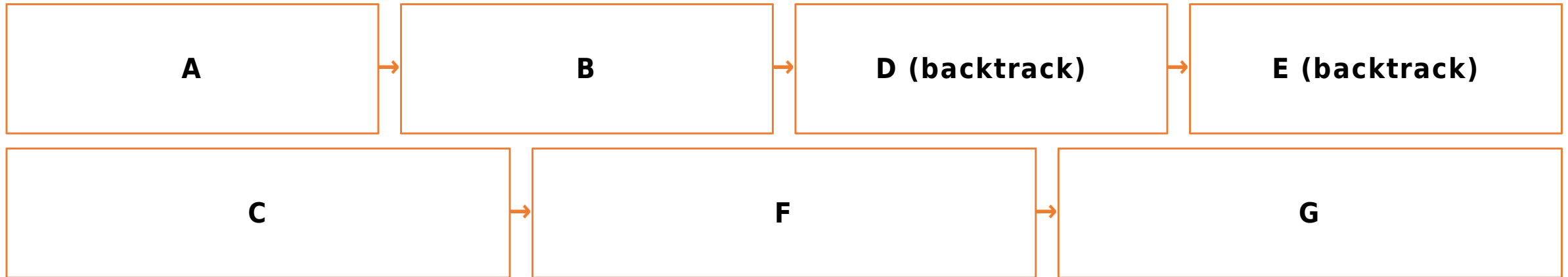


DFS explores deeply: $A \rightarrow B \rightarrow D$. D has no further successor, so it backtracks to E, then returns to $C \rightarrow F \rightarrow G$. DFS does not complete the entire level before going deeper — it follows one branch as far as possible.

Goal = G; left successor selected before right

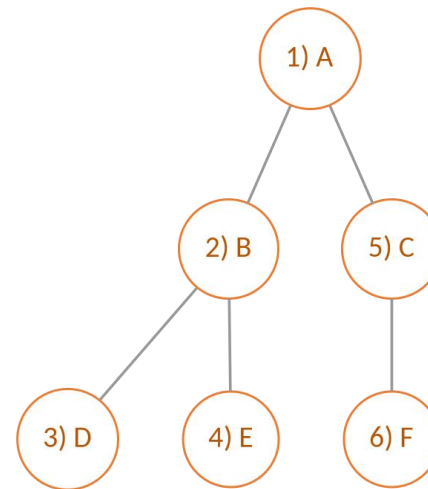
CSE276

DFS Example — Step-by-Step Traversal



Traversal order: A → B → D → E → C → F → G. The algorithm reaches the goal only after exploring the earlier branches in depth-first order.

DFS Traversal Calculation



DFS order

A → B → D → E → C → F

Number of selected nodes

6 nodes

**Max depth before first
backtracking**

A → B → D, reaching depth 2

DFS can reach a deep node quickly without processing all shallower nodes first, but it can be inefficient when it follows a poor branch for a long time.

DFS Properties

1 **Depth-Oriented**

Explores one branch deeply before considering alternatives

2 **Memory Efficient**

Generally stores a smaller active frontier than breadth-first exploration

3 **Backtracking**

Returns to a previous state and explores another branch when needed

4 **Completeness**

Not guaranteed in every space, especially with cycles or unbounded depth

5 **Solution Quality**

Does not guarantee a shortest solution; depends on branch order

DFS Strengths and Limitations

Strengths

- Requires relatively less memory than BFS in many structures
- Can reach deep solutions quickly
- Simple stack-based implementation
- Useful when memory is limited

Limitations

- May follow an unsuitable branch for a long time
- Does not guarantee the shortest solution
- Can fail in unbounded or cyclic spaces without handling
- Performance depends strongly on branch order

Suitable situations: Maze exploration, puzzle search, backtracking problems, exploring large-depth search structures.

BFS vs DFS

Aspect	BFS	DFS
Exploration	Level by level	Branch by branch
Main structure	Queue	Stack
Order	Breadth first	Depth first
Memory	Usually higher	Usually lower
Shortest solution	Minimum depth when step costs are equal	Not guaranteed
Main risk	Large memory usage	Poor/deep branch exploration

BFS asks

Which nearby states should be explored first?

DFS asks

How far can we go along this branch before backtracking?

Choosing BFS or DFS

Prefer BFS when

- The solution is close to the initial state
- Minimum number of steps matters
- The search space fits in memory
- Every step has the same cost

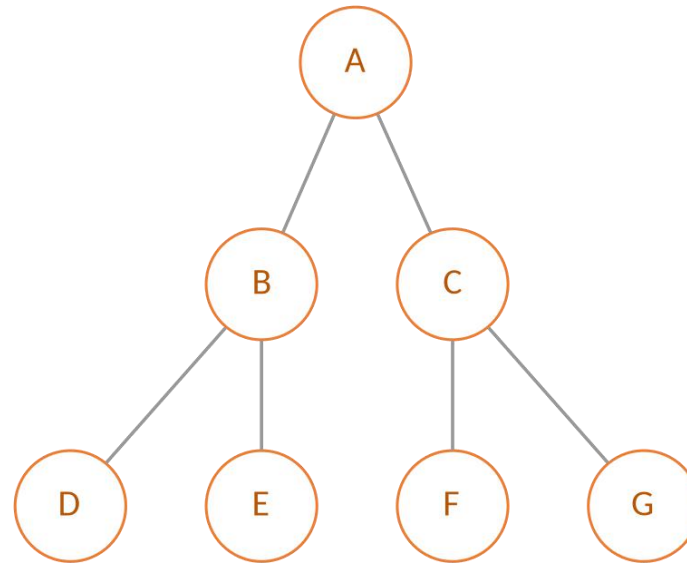
Prefer DFS when

- The solution may be deep
- Memory availability is limited
- Deep exploration is useful
- A minimum-depth solution is not required

Important: Algorithm selection should be based on problem requirements, not on one algorithm being universally superior.

Assume the goal is G

BFS or DFS? — Search Tree Challenge



BFS

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

DFS

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Both eventually reach G, but their exploration effort and memory behavior are different.

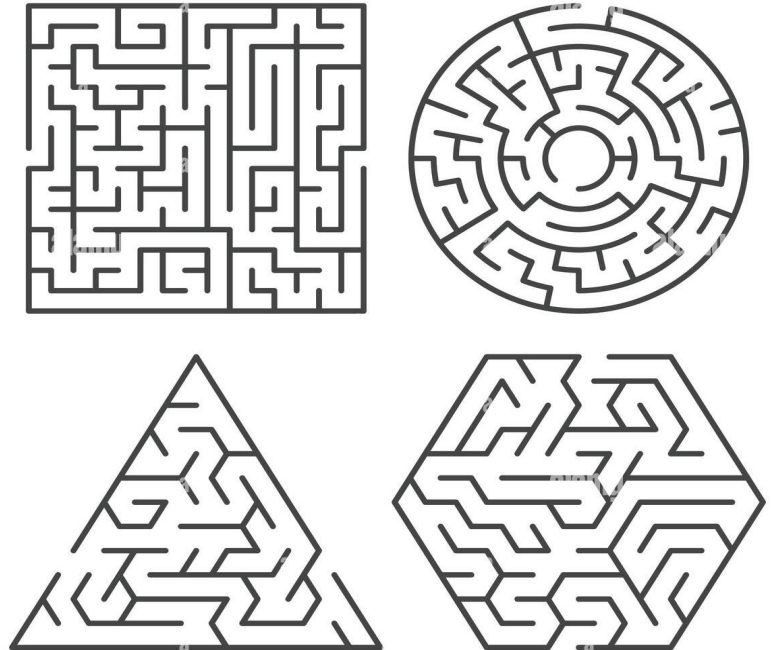
Search Tree Challenge — Maze Search Scenario

Situation

Start position known, exit relatively close, every movement has the same cost, several paths possible

Suitable approach

Breadth First Search



Reason: BFS explores the maze level by level and can find a minimum-step solution when all movements have equal cost. DFS may enter a long incorrect branch before returning and trying another route.

Why Uninformed Search Can Become Expensive

BFS and DFS know the structure of the problem, but they do not know which state is likely to be closer to the goal. As the branching factor increases, the number of possible states can grow rapidly.

- Many states may be irrelevant
- Several paths may lead away from the goal
- The search may spend time on unproductive branches
- Large search spaces increase time and memory needs

BFS and DFS are useful foundations, but many AI problems benefit from additional guidance. That leads to informed search.

Informed Search

Definition: Informed search uses additional information about the problem to guide the search toward promising states.

Uninformed Search

Problem information → explore according to a fixed strategy

Informed Search

Problem information + guiding information → explore promising states first

The major informed-search techniques in this unit are Best First Search, Hill Climbing, and A* Search — all built on the heuristic function.

Heuristic Function

Definition: A heuristic function provides an estimate of the cost or closeness of a state to the goal, commonly written $h(n)$.

n	Current node or state
$h(n)$	Estimated cost from that state to the goal

Example: In route planning, the straight-line distance from the current location to the destination provides a simple estimate. Better guidance can reduce unnecessary exploration.

Good Heuristics

A useful heuristic should provide meaningful guidance without making the search unnecessarily difficult.

1 Relevant

It should relate to the actual goal

2 Informative

It should distinguish promising states from less promising ones

3 Efficient

It should be inexpensive enough to calculate

4 Consistent

Its meaning should match the search objective

Example: For route planning, straight-line distance provides a simple estimate of how far a location is from the destination.

Best First Search

Definition: Best First Search is an informed search technique that selects the most promising node according to an evaluation value, using heuristic information rather than depth or fixed order.

Generate Possible States



Evaluate Them



Select Most Promising

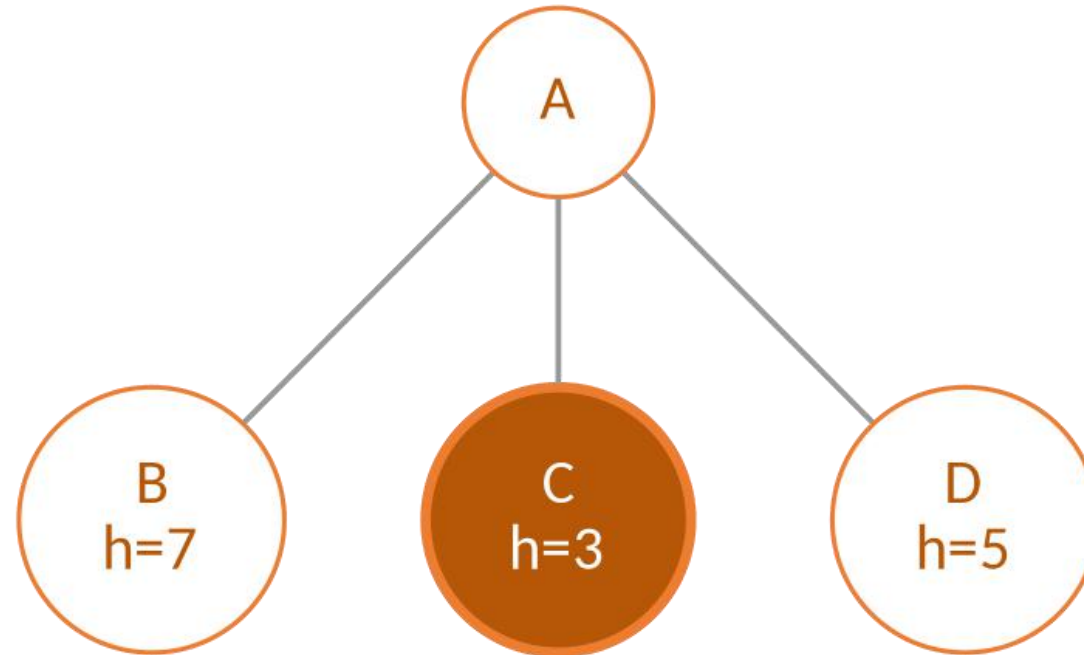


Continue

- Uses heuristic information
- Selects one promising node at each step
- Can reach a goal faster than uninformed search
- Does not necessarily guarantee the best solution

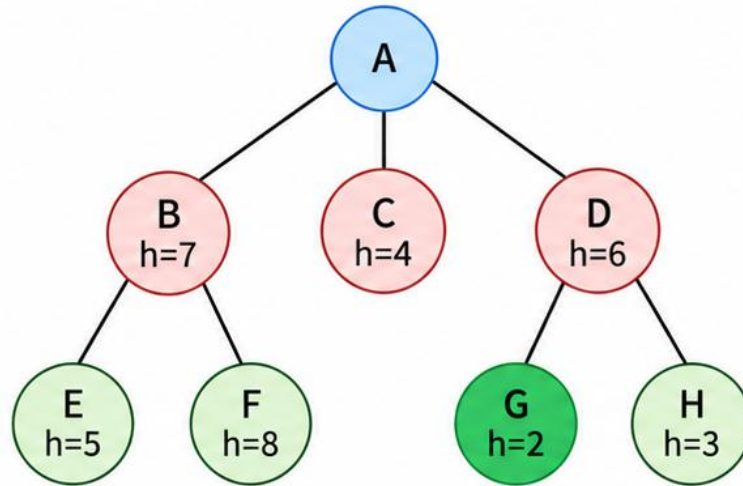
Suppose the heuristic values are $B=7$, $C=3$, $D=5$

Best First Strategy

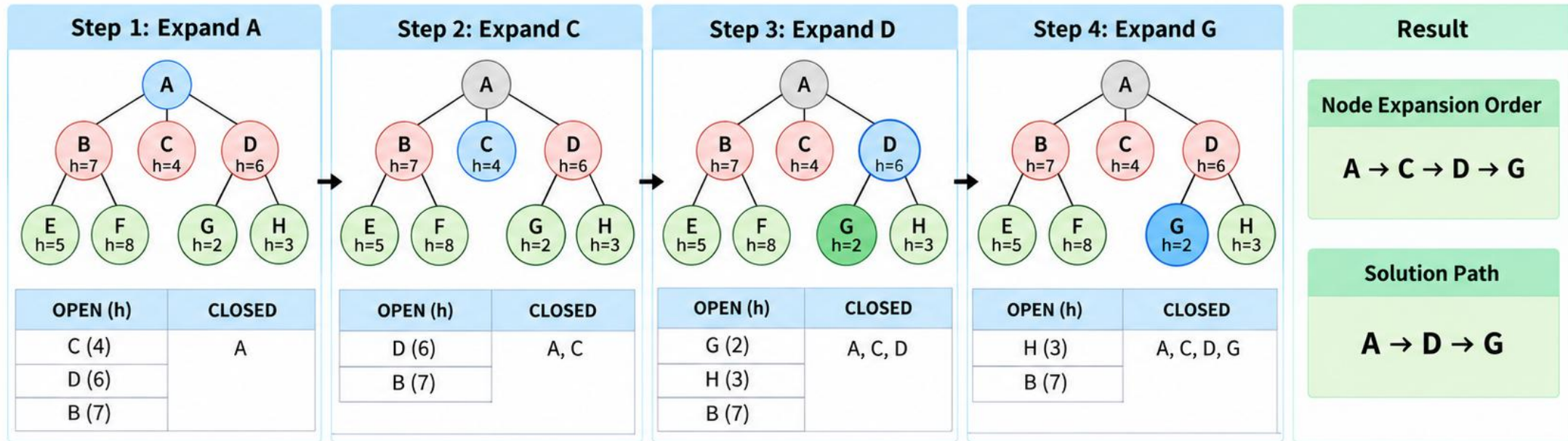


Core rule: Select the available node with the most favorable heuristic value. C is selected first because $h(C) = 3$ is the smallest value. After expanding C, the process continues by selecting the currently most promising state.

Best First Example – Worked Example



Node	$h(n)$
A	–
B	7
C	4
D	6
E	5
F	8
G	2
H	3



Suppose the currently available nodes are

CSE276

Best First Calculation

Node	$h(n)$	Selection
A	9	
B	5	
C	2	Selected
D	7	

Best First Search selects the node with the smallest $h(n)$. $h(C)=2$ is preferred over $h(B)=5$, $h(D)=7$, and $h(A)=9$, so C is chosen for the next expansion. The number is simple — the important part is that $h(n)$ is an estimate used to guide the search, not necessarily the actual cost.

Best First Strengths and Limitations

Strengths

- Uses problem-specific guidance
- Can reduce unnecessary exploration
- Often reaches promising states faster
- Conceptually simple with a useful heuristic

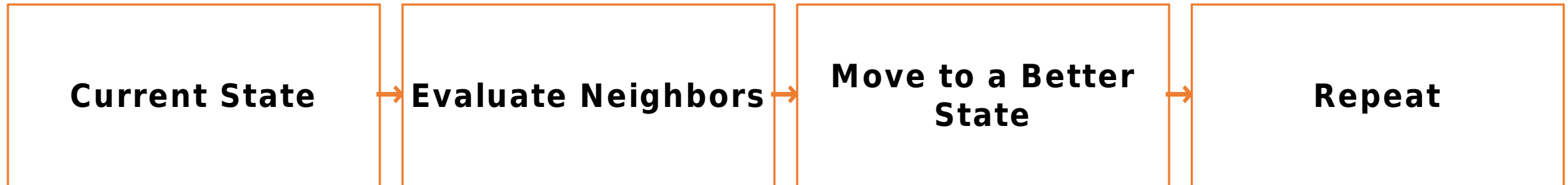
Limitations

- Strongly depends on heuristic quality
- A misleading heuristic can guide toward a poor region
- Does not generally guarantee an optimal solution
- May still explore many unnecessary states

Applications: Route planning, puzzle solving, path finding, intelligent navigation.

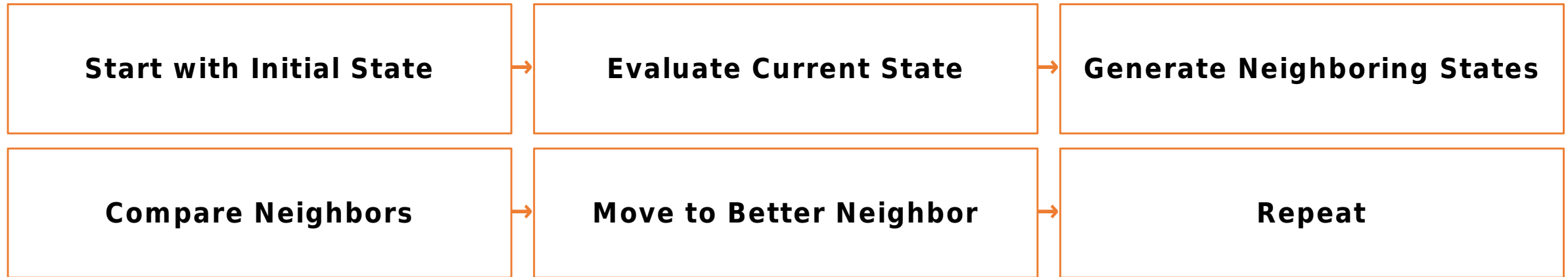
Hill Climbing

Definition: Hill Climbing is an informed local search technique that repeatedly moves from the current state to a neighboring state that appears better according to the evaluation measure.



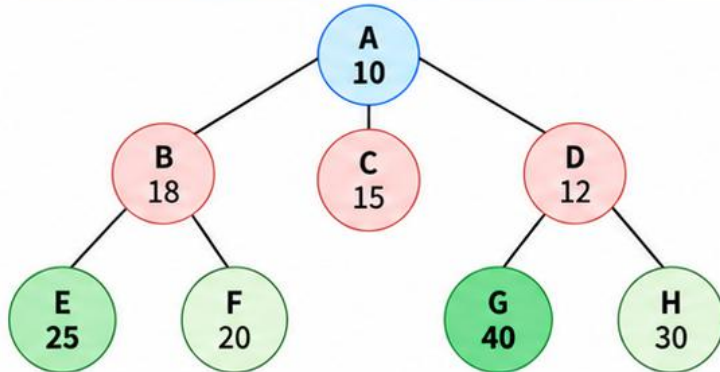
Unlike Best First Search, Hill Climbing focuses strongly on the current state and its neighbors rather than maintaining a broad set of alternatives for global exploration.

Hill Climbing Strategy

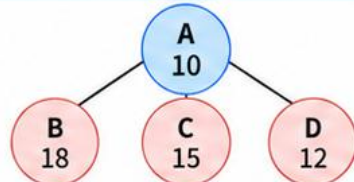


The important characteristic is local decision making. The algorithm does not normally examine the entire search space before each move – it simply asks which nearby state looks better.

Hill Climbing Example – Worked Example

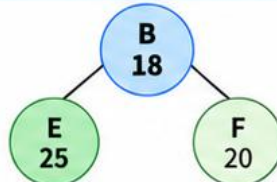
CSE276
State Space Tree (Node Values)

Node Values

Node	Value
A	10
B	18
C	15
D	12
E	25
F	20
G	40
H	30

Step 1: Start at A (10)


Current Node	Neighbors (Values)	Next Move
A (10)	B (18), C (15), D (12)	B (18)

Choose B (highest value 18).

Step 2: Move to B (18)


Current Node	Neighbors (Values)	Next Move
B (18)	E (25), F (20)	E (25)

Choose E (highest value 25).

Step 3: Move to E (25)


Current Node	Neighbors (Values)	Next Move
E (25)	–	–

No neighbor with higher value.
Search stops.

Step 4: Termination

Path Followed

A → B → E

Final Node (Local Maximum)

E (25)

Observation

- Global maximum is G (40).
- Hill climbing stops at E (25), which is a local maximum.
- It does not reach G because it only considers immediate neighbors.

Conclusion

Hill climbing is a greedy local search algorithm.
It may get stuck at a local maximum and fail to reach the global maximum.

Local Maxima, Plateaus and Ridges

1 Local Maximum

A state is better than all its immediate neighbors, but not the best in the complete search space

2 Plateau

Several neighboring states have the same value, so the algorithm cannot easily find a clear improvement direction

3 Ridge

The best direction may require a combination of movements not immediately visible from local neighbors

A locally better move does not always lead to the globally best solution.

Hill Climbing Strengths and Limitations

Strengths

- Simple to understand and implement
- Uses relatively limited search memory
- Can quickly improve a solution
- Avoids maintaining a large set of alternative paths

Limitations

- Can become trapped at a local maximum
- Plateaus can stop progress
- Ridges can make the best direction hard to identify
- Does not guarantee a globally optimal solution

A fast local improvement strategy can be useful even when it cannot guarantee the global optimum.

A* Search — Basic Idea

Definition: A* Search is an informed search algorithm that combines the cost already spent with an estimate of the remaining cost to guide the search.

$f(n)$

$g(n) + h(n)$

$g(n)$

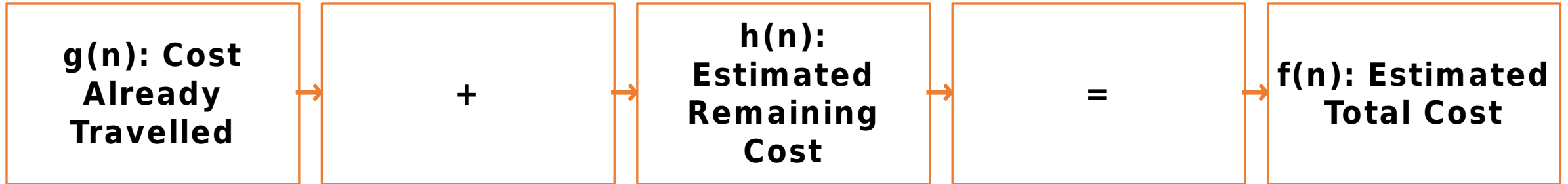
Actual cost from the initial state to node n

$h(n)$

Estimated cost from node n to the goal

Best First Search mainly uses heuristic guidance. A* considers both how much has already been spent and how much more might be required — a more balanced strategy for many path-finding problems.

A* Evaluation



1 **g(n)**
The actual cost from the starting state to the current node

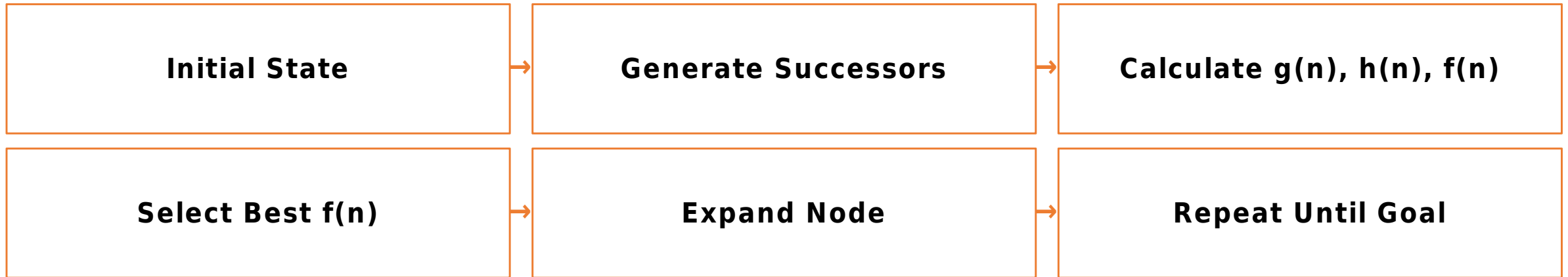
2 **h(n)**
An estimate of the cost from the current node to the goal

3 **f(n)**
The combined value used to decide which node appears most promising

A maintains candidate nodes and repeatedly selects the node with the most favorable $f(n)$*

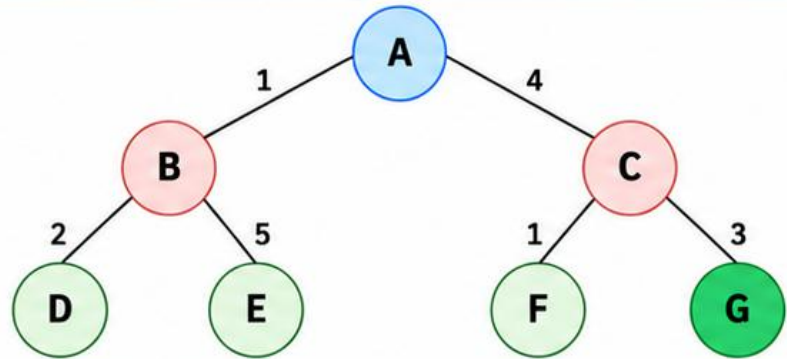
CSE276

A* Working



The important difference from simpler informed search is that A* balances current path cost and estimated future cost.

Graph (Edge Cost)



Heuristic Values $h(n)$

Node	$h(n)$
A	6
B	4
C	2
D	7
E	3
F	1
G	0

Edge Costs

From	To	Cost
A	B	1
A	C	4
B	D	2
C	F	1
C	G	3

Evaluation Function

$$f(n) = g(n) + h(n)$$

Goal Node

G

Step 1: Start with A

Node	$g(n)$	$h(n)$	$f(n)$
A	0	6	6

OPEN: [A(6)]

CLOSED: []

Select A (only node).

Step 2: Expand A

Node	$g(n)$	$h(n)$	$f(n)$
B	1	4	5
C	4	2	6

OPEN: [B(5), C(6)]

CLOSED: [A]

Select B (lowest $f = 5$).

Step 3: Expand B

Node	$g(n)$	$h(n)$	$f(n)$
D	3	7	10
E	6	3	9
C	4	2	6

OPEN: [C(6), E(9), D(10)]

CLOSED: [A, B]

Select C (lowest $f = 6$).

Step 4: Expand C

Node	$g(n)$	$h(n)$	$f(n)$
F	5	1	6
G	7	0	7
E	6	3	9
D	3	7	10

OPEN: [F(6), G(7), E(9), D(10)]

CLOSED: [A, B, C]

Select F (lowest $f = 6$).

Step 5: Expand F

Node	$g(n)$	$h(n)$	$f(n)$
G	8	0	8
E	6	3	9

OPEN: [G(8), E(9), D(10)]

CLOSED: [A, B, C, F]

Select G (lowest $f = 8$) → Goal found.

Final Answer

Node Expansion Order	A → B → C → F → G
Solution Path	A → C → F → G
Total Cost	8 (A → C: 4, C → F: 1, F → G: 3)
Goal Node	G

$g(n)$ Calculation Along Solution Path

$$\begin{aligned} g(A) &= 0 \\ g(C) &= g(A) + 4 = 4 \\ g(F) &= g(C) + 1 = 5 \\ g(G) &= g(F) + 3 = 8 \\ f(G) &= g(G) + h(G) = 8 + 0 = 8 \end{aligned}$$

A* Properties

1 Informed

A* uses heuristic information to guide the search

2 Cost-Aware

It considers the actual cost already travelled through $g(n)$

3 Goal-Oriented

The heuristic $h(n)$ estimates the remaining cost to the goal

4 Complete Under Standard Conditions

With appropriate assumptions, A* can find a solution when one exists

5 Optimal Under Appropriate Conditions

When the heuristic satisfies suitable conditions, A* can find an optimal solution

Important condition: The quality of the result depends strongly on the heuristic function and the cost model used by the problem.

A* Strengths, Limitations and Applications

Strengths

- Combines actual path cost with heuristic guidance
- Can find high-quality paths efficiently
- More informed than uninformed search
- Flexible because the heuristic can reflect problem knowledge

Limitations

- Can require significant memory
- Performance depends on heuristic quality
- A poor heuristic may reduce efficiency
- Computational cost can become high for very large spaces

Applications: Route planning, robot path planning, maze solving, puzzle solving, game and navigation systems.

Search Algorithm Comparison

Algorithm	Guidance	Main Strategy	Structure	Key Strength	Major Limitation
BFS	No heuristic	Level by level	Queue	Minimum-depth solution (equal costs)	High memory
DFS	No heuristic	Deep branch first	Stack	Lower memory in many cases	Can follow poor deep branches
Best First	Heuristic	Most promising heuristic state	Priority-based	Guided exploration	May not give optimal solution
Hill Climbing	Heuristic	Improve current state	Local selection	Simple, memory-efficient	Local maxima / plateaus
A*	Heuristic + path cost	Lowest $f(n)$	Priority-based	High-quality guided search	Can be memory-intensive

Uninformed: BFS, DFS. Informed: Best First, Hill Climbing, A*. The algorithms differ mainly in how they decide which state deserves attention next.

Search Applications and Case Study

1 Route Planning

Search can identify possible paths between locations

2 Maze Solving

BFS or DFS can explore possible movements toward an exit

3 Robot Path Planning

Search can determine a route through a structured environment

4 Puzzle Solving

Hill Climbing and A* can explore possible puzzle configurations

Practical selection: There is no single best search algorithm for every problem. The choice depends on search-space size, solution depth, memory availability, heuristic availability, and required solution quality.

From Search to Knowledge

We have completed the first part of Unit II — how an AI system explores possible states and finds a solution, using BFS, DFS, heuristic functions, Best First Search, Hill Climbing, and A Search.*

An intelligent system often needs more than a search strategy. It also needs to store useful knowledge about the world and use that knowledge to reach conclusions.

Search



Knowledge



Representation



Reasoning

Why Knowledge Representation?

An AI system may receive a large amount of information, but information becomes useful only when the system can organize and use it.

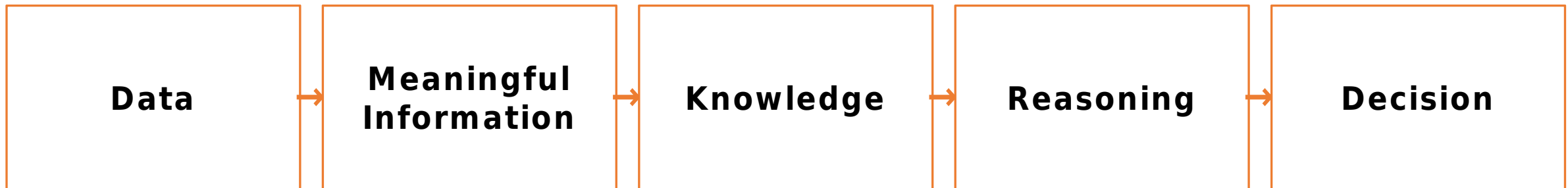
For intelligent reasoning, a system may need to represent

- Facts about objects and situations
- Relationships between objects
- Rules about how situations are related
- General concepts and their properties
- Knowledge needed for decision making

Example: A medical system may need knowledge such as: fever is associated with illness, and certain symptom combinations may support a diagnosis. The system needs a structured way to store such knowledge so it can later reason from it.

Data, Information and Knowledge

Data	Raw observations without interpretation — e.g. 39.2°C
Information	Data interpreted in context — e.g. the patient's temperature is 39.2°C
Knowledge	Information plus relationships or rules — e.g. a high temperature can signal certain illnesses



Knowledge in an AI system can describe different aspects of a problem

CSE276

Knowledge in AI

Facts

- 1 Information considered true in the domain — “Paris is in France”

Relationships

- 2 Connections between entities — “Student A studies at University X”

Properties

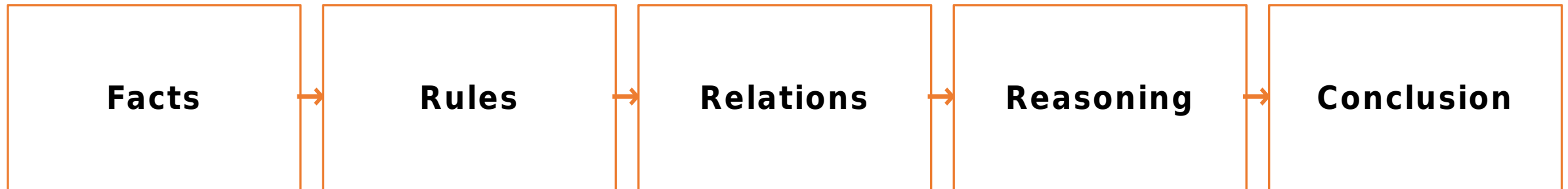
- 3 Characteristics of an object — “A car has four wheels”

Rules

- 4 Conditions linking facts to conclusions — “If a vehicle has no fuel, it cannot run”

Knowledge Base

Definition: A knowledge base is a structured collection of facts, relationships, rules, or other represented knowledge that an AI system can use for reasoning.



Important distinction: Knowledge Base = represented knowledge. Reasoning = using that knowledge.

Inference and Reasoning

Definition: Inference is the process of deriving conclusions from available knowledge.

Fact

Ravi has fever

Rule

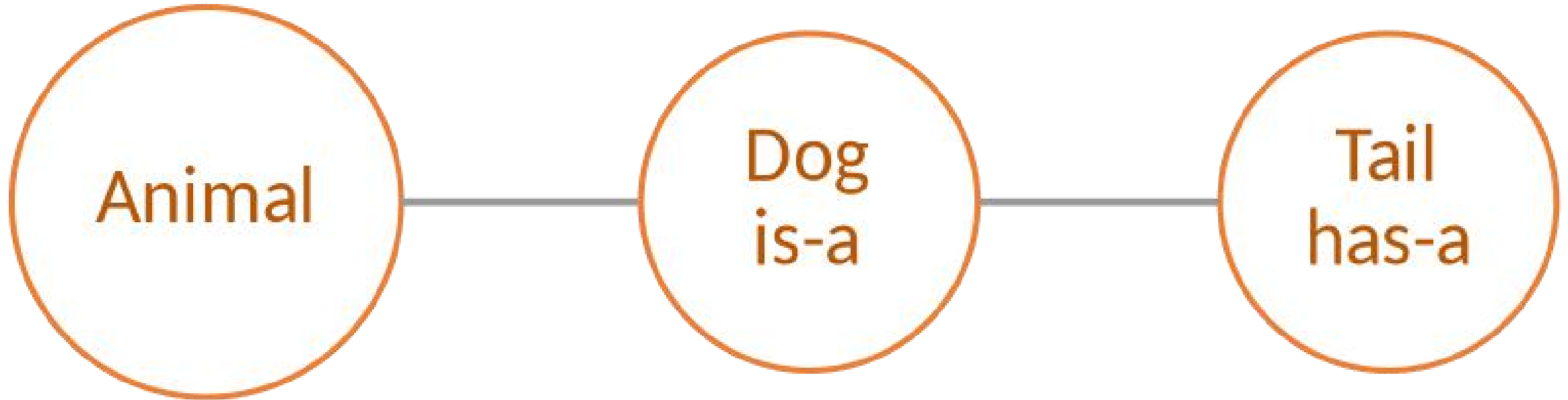
If a person has fever, then fever is a symptom

Conclusion

Ravi has a symptom associated with fever

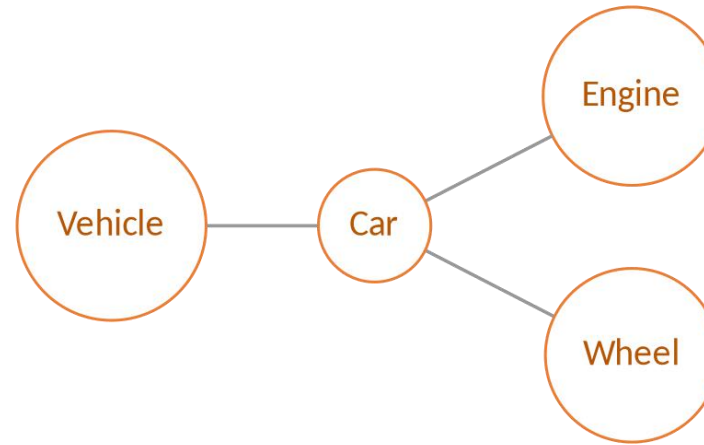
Representation stores what the system knows. Inference determines what can be concluded from that knowledge.

Semantic Networks



Definition: A semantic network represents knowledge as a graph of concepts and relationships, using nodes for objects or concepts and links for relationships between them. Knowledge is represented as connected concepts rather than isolated facts.

Nodes and Links



Nodes represent

- Objects
- Concepts
- People / places
- Events / categories

Links represent

- is-a
- has-a
- part-of
- located-in

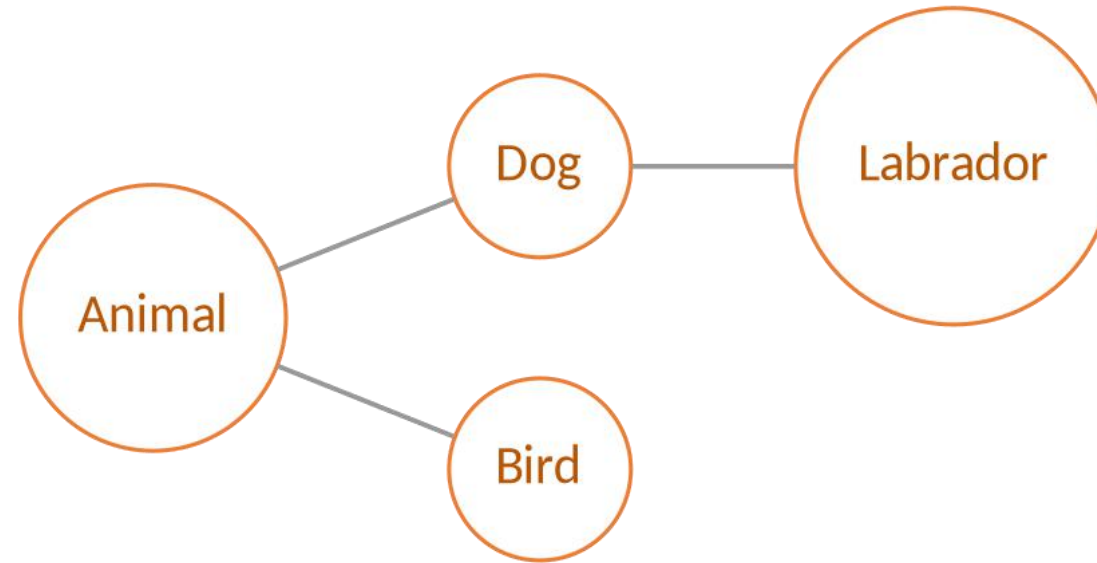
Without relationships, the system has a collection of concepts. With relationships, the system has structured knowledge.

Relationships in Semantic Networks

Relationship	Meaning	Example
is-a	Category or class membership	Car → is-a → Vehicle
has-a	Possession or composition	Car → has-a → Engine
part-of	Belongs to a larger structure	Engine → part-of → Car
located-in	Location	Library → located-in → Campus

The relationship itself carries meaning — “Dog → is-a → Animal” tells us more than simply storing “Dog” and “Animal” as two separate concepts.

Inheritance in Semantic Networks



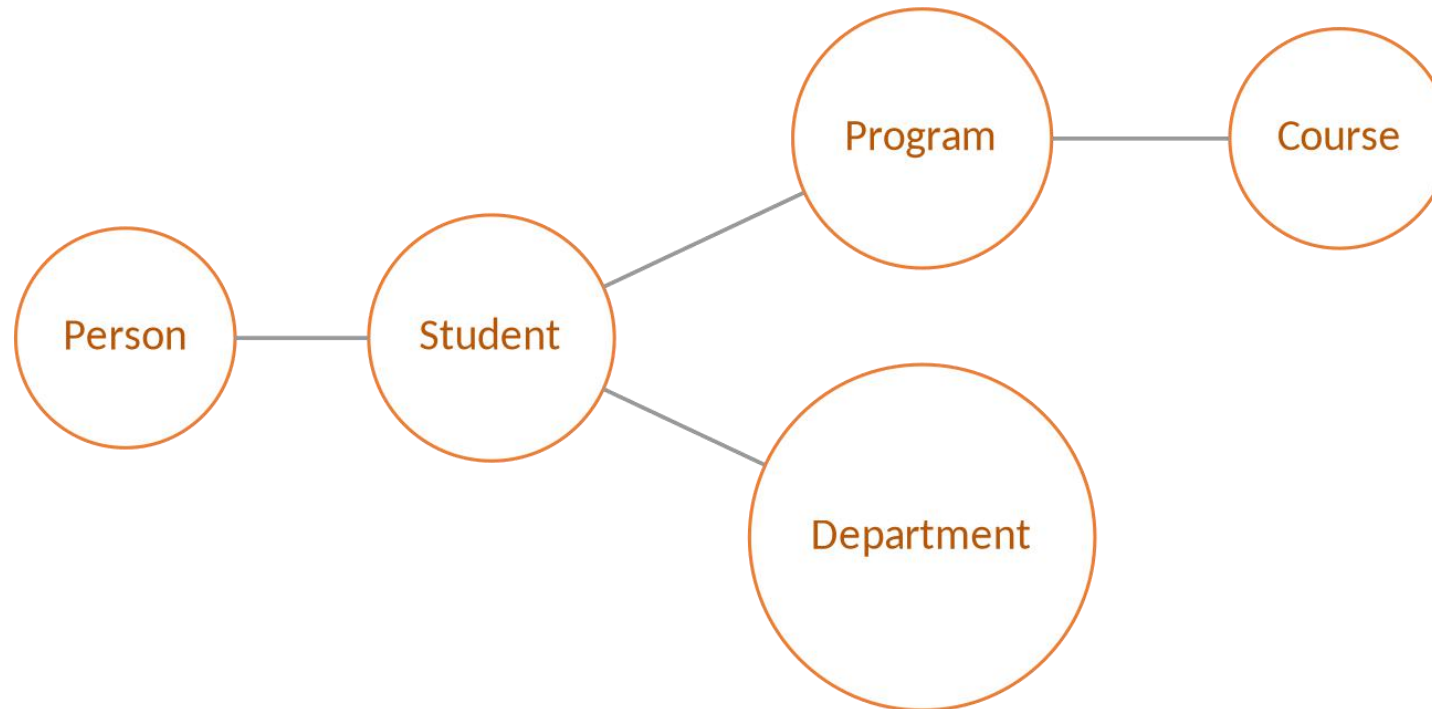
General Concept

Shared Property

Specific Concept

If Animal → can breathe, then Dog can inherit that property through Dog → is-a → Animal. Inheritance reduces unnecessary repetition because common properties do not need to be stored independently for every concept.

Semantic Network Example



A Student is a Person, studies a Program, and belongs to a Department; a Program has Courses. If Ravi is a Student, and Student is a Person, the system can infer that Ravi is a Person — demonstrating how structural relationships support reasoning.

Semantic Networks — Strengths and Limitations

Strengths

- Easy to visualize
- Clearly represents relationships
- Supports inheritance
- Useful for hierarchical domains

Limitations

- Complex relationships can be hard to manage
- Large networks may be difficult to interpret
- Formal reasoning may need extra mechanisms
- Uncertainty is not naturally represented

Semantic networks are useful when the main requirement is to represent concepts and relationships in a structured graphical form. This prepares us for Frames.

Frames

Student	
Name	
Program	
Semester	
Department	
Enrollment Status	

Definition: A frame is a structured representation used to describe an object, concept, or situation through a collection of attributes and their values. Semantic Network emphasizes relationships; Frame emphasizes structured properties of one entity.

Frame Structure

Frame: Student	
Name	Ravi
Program	CSE
Semester	3
Department	Computer Science

Frame	Represents the object or concept
Slots	Represent attributes or properties
Values	Represent the information stored in those slots

Slots and Values

Slot	Value
Name	Ravi
Program	CSE
Semester	3
Department	Computer Science
Status	Active

Instead of storing every fact independently, related information can be organized under one entity. This makes the representation compact and easy to access, and frames can also support default and inherited values.

A more specific frame can reuse relevant information from a general frame

Frame Example

Vehicle

- Type → Car
- Fuel → Petrol
- Wheels → 4
- Color → White

Car (inherits Vehicle)

- Model → Sedan
- Seats → 5
- (plus all Vehicle properties)

**Vehicle → general properties. Car → Vehicle properties + Car-specific properties.
This makes frames useful for hierarchical and structured knowledge.**

Frame Inheritance

Vehicle (general)

Wheels → 4
Engine → Present

Car (inherits Vehicle)

Fuel → Petrol
(plus Wheels and Engine from Vehicle)

Benefits

- Reduces duplication
- Organizes knowledge hierarchically
- Supports default information
- Makes related concepts easier to manage

Frames — Strengths and Limitations

Strengths

- Organize related properties in one structure
- Easy to represent objects and situations
- Support inheritance
- Reduce repeated information

Limitations

- Less natural for complex relationships between many entities
- Large frame hierarchies can be hard to manage
- Rules and dynamic reasoning may need extra mechanisms
- Basic frames do not naturally represent uncertainty

The next step is to compare Frames directly with Semantic Networks and understand when each representation is more suitable.

Semantic Networks vs Frames

Aspect	Semantic Networks	Frames
Main focus	Concepts and relationships	Objects and their properties
Basic structure	Nodes and links	Frame with slots and values
Strength	Clearly shows relationships	Organizes structured information
Inheritance	Through connected concepts	Through frame hierarchy
Best suited for	Relationship-rich knowledge	Object- or situation-based knowledge

Semantic Network asks

How are these concepts connected?

Frame asks

What information describes this object or situation?

Choosing a Representation

Use a Semantic Network when

- Relationships between concepts are central
- Hierarchies are important
- Connections need to be visualized
- Inheritance through relationships is useful

Use Frames when

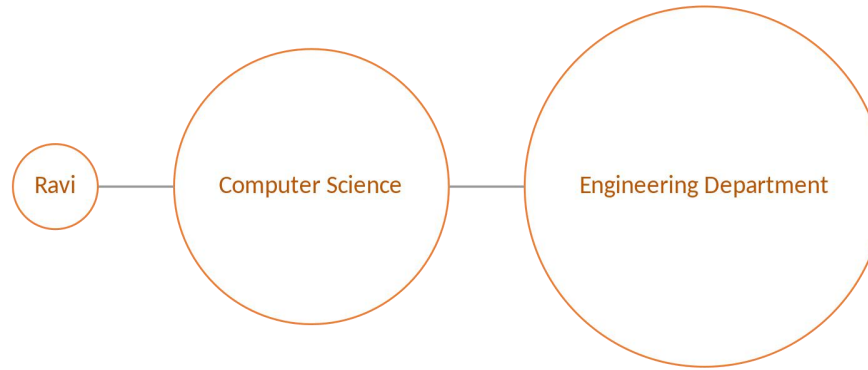
- Objects have a fixed set of properties
- Information can be organized into slots
- Default values are useful
- Similar objects share a common structure

The choice depends on how the knowledge itself is structured.

Ravi is a student who studies Computer Science, in the Engineering Department, semester 3, active enrollment

CSE276

Build the Representation



Student	
Name	Ravi
Program	CSE
Semester	3
Status	Active

The goal is not to choose one universally correct representation. It is to recognize which representation makes the required knowledge clearer and easier to use.

Production Systems

Definition: A production system represents knowledge mainly through rules that specify what should happen when particular conditions are satisfied: IF condition THEN action or conclusion.



Example: IF temperature is high AND engine warning is present THEN inspect the cooling system.

IF-THEN Rules

Example 1

IF temperature is high THEN possible fever

Example 2

IF fever AND cough are present THEN consider respiratory illness

Conditions

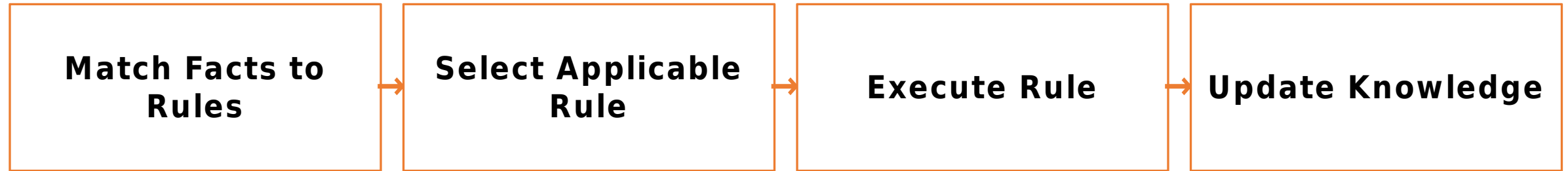
Facts that must be satisfied

Conclusion / Action

Result produced when the rule fires

Rules do not simply store facts — they describe how the system can respond when particular conditions are satisfied.

Production Cycle

**Facts**

Fever = Yes

Rule

IF Fever = Yes THEN Fever-Symptom = Yes

The rule fires and adds a new conclusion. This process can continue as newly derived information makes other rules applicable.

Production Example

Facts

Engine does not start, battery indicator is low, headlights are weak

Rule 1: Battery low → possible battery problem

Rule 2: Weak headlights → check battery

Rule 3: Battery problem suspected → recommend inspection

The system converts facts into conclusions or actions through explicit rules.

Production Systems – Strengths and Limitations

Strengths

- Rules are easy to understand
- Domain knowledge can be represented explicitly
- Individual rules can be modified independently
- Reasoning steps can be traced through fired rules

Limitations

- Large rule sets can become difficult to manage
- Conflicting rules may require a selection strategy
- New situations may require additional rules
- Complex uncertainty is difficult to express

Suitable applications: Decision support, diagnosis, configuration, troubleshooting, rule-based automation.

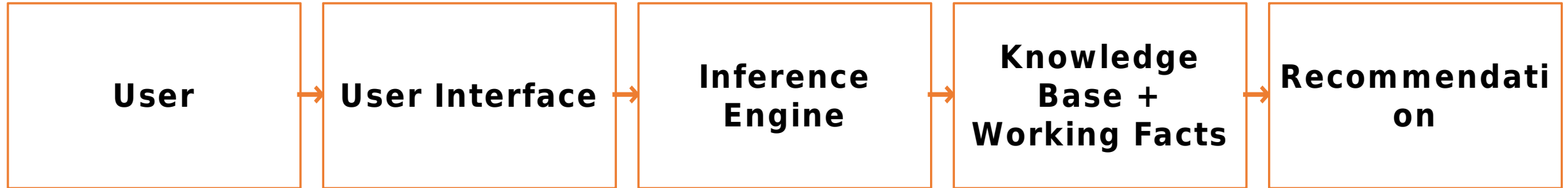
Expert Systems

Definition: An expert system is an AI system designed to use stored domain knowledge and reasoning mechanisms to provide decisions or advice similar to a human expert's problem-solving process within a specific domain.



Typical domains: Healthcare, finance, equipment diagnosis, technical support, decision support.

Expert System Architecture



1 Knowledge Base
Stores domain-specific knowledge

2 Inference Engine
Applies reasoning mechanisms to available facts and rules

3 Working Information
Contains the facts relevant to the current problem

4 User Interface
Allows the user to provide information and receive recommendations

Knowledge Base and Inference Engine

Knowledge Base

Contains domain knowledge: facts, rules, relationships.

Example: fever + cough → possible respiratory condition.

Inference Engine

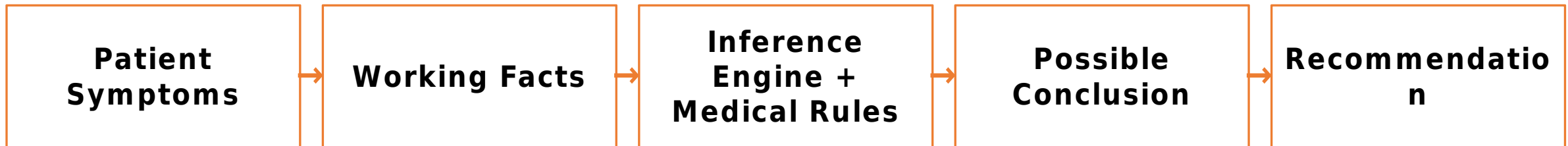
Uses the knowledge base together with current facts to derive conclusions.

An expert system requires both. Knowledge without reasoning cannot produce useful conclusions; reasoning without relevant knowledge has little domain value.

A simplified medical diagnosis example

Medical Expert System

Symptoms	Fever, Cough, Fatigue
Rule 1	IF fever AND cough THEN consider respiratory infection
Rule 2	IF fever AND fatigue THEN recommend further evaluation



The system does not replace a healthcare professional. Its purpose here is to demonstrate how domain knowledge and logical rules can support decision making.

Expert Systems – Strengths, Limitations and Applications

Strengths

- Preserve specialized domain knowledge
- Provide consistent rule-based reasoning
- Can support decision making
- Reasoning steps can often be traced

Limitations

- Usually restricted to a specific domain
- Knowledge must be created and maintained
- Difficult to handle cases outside encoded knowledge
- Rule conflicts can create reasoning problems

Applications: Medical decision support, equipment diagnosis, financial advisory systems, technical troubleshooting.

Build an Expert Rule

Facts

Attendance is low, assignment work is incomplete, recent performance has declined

**Rule 1: Low attendance →
flag concern**

**Rule 2: Concern + decline
→ recommend follow-up**

**Rule 3: Incomplete work →
recommend support**

Facts → Rules → Conclusions → Recommendation. This shows how an expert system converts domain knowledge into structured decision support.

Why Logic in AI?

Rules and knowledge need a precise way to express relationships and conclusions. Logic provides formal structures for representing statements and deriving conclusions.

It helps an AI system

- Represent facts precisely
- Express relationships between statements
- Determine whether conclusions follow from information
- Perform structured reasoning
- Reduce ambiguity in knowledge representation

Two logical forms in this unit are Propositional Logic and First-Order Predicate Logic — both support formal reasoning, but at different levels of detail.

Propositions

Definition: A proposition is a declarative statement that can be evaluated as either True or False.

P

"The server is running."

Q

"The network is available."

P AND Q

Both conditions must be true

Important limitation: A basic proposition represents the statement as a whole — it does not directly represent individual objects, properties, or relationships inside the statement. That leads to the need for predicate logic.

Logical Operators – AND, OR and NOT

P	Q	P AND Q
T	T	T
T	F	F
F	T	F
F	F	F

$P \wedge Q$ (AND)

True only when both P and Q are true

$P \vee Q$ (OR)

True when at least one proposition is true

$\neg P$ (NOT)

Reverses the truth value of P

Let P = Server is running, Q = Network is available. Then $P \wedge Q$ means: Server is running AND network is available.

P: Server is running. Q: Network is available. Evaluate $P \wedge Q$

CSE276

Truth Tables and Reasoning

Case	P	Q	$P \wedge Q$
1	True	True	True
2	True	False	False
3	False	True	False

The truth table provides a systematic way to determine the result of a logical expression. Formal logic allows an AI system to determine whether a conclusion follows from represented facts and logical relationships.

First-Order Predicate Logic

Propositional logic treats an entire statement as one unit — “Ravi studies Computer Science” is simply one proposition. But AI often needs more detail.

We may want to represent

- Individual objects
- Properties of objects
- Relationships between objects
- General rules
- Statements about all or some objects

Example: Instead of treating the whole statement as one symbol, we can represent **Studies(Ravi, CSE)** — making the objects and their relationship explicit.

Predicates and Quantifiers

Student(Ravi)

Ravi is a student

Studies(Ravi, CSE)

A relationship between Ravi and CSE

$\forall$ Universal — “for all”

$\forall x \text{ Student}(x) \rightarrow \text{HasID}(x)$

Every student has an ID.

$\exists$ Existential — “there exists”

$\exists x \text{ Student}(x) \wedge \text{Studies}(x, AI)$

At least one student studies AI.

Predicate logic can represent objects, properties, relationships and quantified statements, making it more expressive than basic propositional logic.

Reasoning Under Uncertainty

Real-world information is often incomplete, noisy, or uncertain. An AI system may not know that a conclusion is definitely true — it may need to reason using probabilities or degrees of belief.

Deterministic reasoning

Given facts → certain conclusion

Uncertain reasoning

Evidence → probable conclusion

Bayes' theorem provides a mathematical way to update the probability of a hypothesis when new evidence becomes available.

Bayes' Theorem and Example

Formula: $P(H | E) = P(E | H) \times P(H) / P(E)$

P(Disease)	0.01
P(Positive Disease)	0.90
P(Positive No Disease)	0.05
P(Positive)	$(0.90 \times 0.01) + (0.05 \times 0.99) = 0.0585$
P(Disease Positive)	$(0.90 \times 0.01) / 0.0585 \approx 15.4\%$

New evidence changes the probability of a hypothesis rather than automatically making the hypothesis certain. This supports medical decision support, fraud detection, classification, and risk assessment.